

Module 1

Initiation PyTorch, MLP & CNN

Fondements mathématiques — loss, optimiseurs, métriques

Cadre mathématique du Deep Learning

Du tenseur au gradient — du neurone au CNN



Tenseurs

Autograd, graphe



Backprop

Chain rule



Optimiseurs

SGD, Adam



Métriques

F1, AUC, ROC

Bassem Ben Hamed

bassem.benhamed@enetcom.usf.tn

ENETCOM — Université de Sfax

Présentation 01 — Module 1

1. Fondations PyTorch : tenseur & autograd
2. Le neurone et le MLP
3. Fonctions de coût (Loss)
4. Backpropagation
5. Optimiseurs et mise à jour des poids
6. Régularisation & normalisation
7. Réseaux convolutifs (CNN)
8. Métriques de performance
9. Synthèse & transition

Fondations PyTorch : tenseur & autograd

Le tenseur — objet mathématique central

Définition

Un **tenseur** d'ordre n est un tableau multidimensionnel de réels : $\mathbf{T} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_n}$. PyTorch généralise les vecteurs ($n=1$), matrices ($n=2$), et tenseurs d'ordre supérieur (images $n=3$, batch d'images $n=4$).

Opérations algébriques fondamentales :

- Produit scalaire : $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$
- Produit matriciel : $(\mathbf{AB})_{ij} = \sum_k A_{ik} B_{kj}$
- Produit tensoriel : $(\mathbf{A} \otimes \mathbf{B})_{ijkl} = A_{ij} B_{kl}$
- **Broadcasting** : alignement implicite des dimensions

```
import torch
x = torch.randn(32, 3, 224, 224)  # batch images
W = torch.randn(3, 64, 3, 3)      # conv kernel
y = torch.matmul(x, W)             # (m,k)@(k,n)
z = x + torch.tensor([1,2,3])     # broadcasting
x.shape, x.device, x.dtype
```

Autograd — différentiation automatique

Principe

PyTorch construit dynamiquement un **graphe de calcul** (DAG) où chaque nœud est un tenseur et chaque arête une opération différentiable. La rétropropagation applique la **règle de la chaîne** pour calculer $\nabla_{\theta} \mathcal{L}$.

Règle de la chaîne (composition) :

$$\text{si } y = f(u), \quad u = g(x) \Rightarrow \frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial x}$$

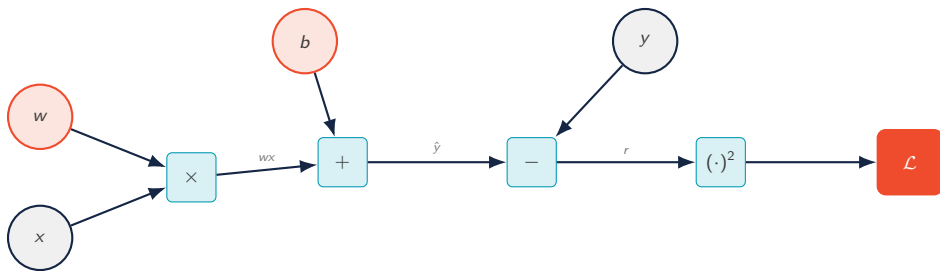
Cas multivarié : si $y = f(u)$ et $u = g(x)$,

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \frac{\partial u}{\partial x} \quad (\text{produit de matrices jacobiennes})$$

```
x = torch.tensor([2.0], requires_grad=True)
y = x**2 + 3*x + 1           # forward
y.backward()                 # backprop
print(x.grad)                # dy/dx = 2x + 3 = 7
```

- `requires_grad=True` active le suivi
- `.backward()` parcourt le DAG en sens inverse
- `torch.no_grad()` désactive le graphe (inférence)
- `.detach()` extrait un tenseur du graphe

Graphe de calcul — exemple $\mathcal{L} = (wx + b - y)^2$



→ Forward

$$\hat{y} = wx + b, \quad r = \hat{y} - y, \quad \mathcal{L} = r^2$$

← Backward (chain rule)

$$\frac{\partial \mathcal{L}}{\partial w} = 2r \cdot x, \quad \frac{\partial \mathcal{L}}{\partial b} = 2r$$

Pipeline d'entraînement PyTorch

Les 5 étapes canoniques

1. Charger les données via `Dataset` & `DataLoader`
2. Définir le modèle : héritage de `nn.Module`
3. Choisir la *loss* et l'optimiseur
4. Itérer (epoch / batch) : forward → loss → backward → step
5. Évaluer sur le set de validation / test

```
for epoch in range(num_epochs):  
    model.train()  
    for X, y in train_loader:  
        X, y = X.to(device), y.to(device)  
        optimizer.zero_grad()           # 1  
        y_hat = model(X)                 # 2 forward  
        loss = criterion(y_hat, y)       # 3 loss  
        loss.backward()                  # 4 backward  
        optimizer.step()                  # 5 update
```



Réflexe

Toujours appeler `optimizer.zero_grad()` avant `loss.backward()` pour éviter l'accumulation des gradients.

Le neurone et le MLP

Le neurone artificiel — formulation

Modèle du perceptron (McCulloch & Pitts, 1943)

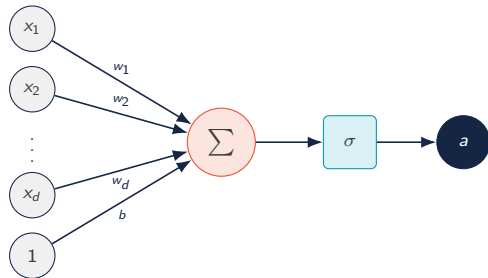
Pour un vecteur d'entrée $\mathbf{x} \in \mathbb{R}^d$, poids $\mathbf{w} \in \mathbb{R}^d$, biais $b \in \mathbb{R}$:

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^d w_i x_i + b, \quad a = \sigma(z)$$

- z : *pré-activation* (combinaison linéaire)
- σ : *fonction d'activation* non linéaire
- a : *activation* (sortie du neurone)

Théorème d'approximation universelle (Cybenko, 1989)

Un MLP à une couche cachée et activations non linéaires peut approximer toute fonction continue sur un compact de $\mathbb{R}^d$ avec une précision arbitraire.



Un neurone = produit scalaire + activation

Fonctions d'activation — formules & dérivées

Activation	$\sigma(z)$	$\sigma'(z)$	Usage
Sigmoïde	$\frac{1}{1 + e^{-z}}$	$\sigma(z)(1 - \sigma(z))$	Classif. binaire (sortie)
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - \tanh^2(z)$	Couches cachées (RNN)
ReLU	$\max(0, z)$	$\mathbf{1}_{z>0}$	Couches cachées (default)
Leaky ReLU	$\max(\alpha z, z), \alpha=0.01$	α si $z<0$, sinon 1	Évite <i>dying ReLU</i>
GELU	$z \Phi(z)$ (Φ : CDF norm.)	$\Phi(z) + z \varphi(z)$	Transformers, ViT
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	$s_i(\delta_{ij} - s_j)$	Classif. multi-classes

Softmax produit une distribution de probabilité : $\sum_i s_i = 1, s_i \in [0, 1]$.

Architecture MLP — propagation avant

Notation pour un MLP à L couches

$$\mathbf{a}^{(0)} = \mathbf{x} \in \mathbb{R}^{d_0}$$

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{W}^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}, \quad \mathbf{b}^{(\ell)} \in \mathbb{R}^{d_\ell}$$

$$\mathbf{a}^{(\ell)} = \sigma^{(\ell)}(\mathbf{z}^{(\ell)}), \quad \ell = 1, \dots, L$$

$$\hat{\mathbf{y}} = \mathbf{a}^{(L)}$$

Comptage des paramètres :

$$\#\theta = \sum_{\ell=1}^L (d_\ell \cdot d_{\ell-1} + d_\ell)$$

Exemple : MLP $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$:

$$784 \cdot 256 + 256 + 256 \cdot 128 + 128 + 128 \cdot 10 + 10 = 235\,146$$

```
class MLP(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.net = nn.Sequential(  
            nn.Linear(784, 256), nn.ReLU(),  
            nn.Linear(256, 128), nn.ReLU(),  
            nn.Linear(128, 10))  
    def forward(self, x):  
        return self.net(x)
```

Fonctions de coût (Loss)

Fonctions de coût — panorama mathématique

Principe général

Une **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}; \theta)$ mesure l'écart entre prédiction et cible. L'entraînement résout $\theta^* = \arg \min_{\theta} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}} [\mathcal{L}(f_{\theta}(\mathbf{x}), \mathbf{y})]$, approximé par la *moyenne empirique* sur le batch.

Tâche	Formule (sur N exemples)	PyTorch
Régression (MSE)	$\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$	<code>nn.MSELoss()</code>
Régression (MAE)	$\frac{1}{N} \sum_{i=1}^N \hat{y}_i - y_i $	<code>nn.L1Loss()</code>
Régression (Huber)	$\rho_{\delta}(\hat{y}_i - y_i)$ quadratique si $ r \leq \delta$, linéaire sinon	<code>nn.SmoothL1Loss()</code>
Classif. binaire	$-\frac{1}{N} \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$	<code>nn.BCEWithLogitsLoss()</code>
Classif. K -class	$-\frac{1}{N} \sum_i \sum_{k=1}^K y_{ik} \log \hat{y}_{ik}$ (CE)	<code>nn.CrossEntropyLoss()</code>
KL divergence	$\sum_k p_k \log(p_k / q_k)$	<code>nn.KLDivLoss()</code>

Cross-Entropy — dérivation complète

Softmax + Cross-Entropy

Pour K classes, logits $\mathbf{z} \in \mathbb{R}^K$, label one-hot $\mathbf{y}$:

$$\hat{y}_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^K y_k \log \hat{y}_k$$

Gradient remarquable

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_k} = \hat{y}_k - y_k$$

(forme très simple — d'où la stabilité numérique)

Astuce log-sum-exp (stabilité)

$$\log \sum_j e^{z_j} = m + \log \sum_j e^{z_j - m}, \quad m = \max_j z_j$$

`nn.CrossEntropyLoss` combine `LogSoftmax` + `NLLLoss` $\Rightarrow$ **ne pas** appliquer Softmax avant !

💡 Bonne pratique

Passer directement les *logits* à `CrossEntropyLoss()` pour la stabilité numérique et la performance.

Backpropagation

Backpropagation — principe général

Objectif

Calculer efficacement $\nabla_{\theta} \mathcal{L}$ pour tous les paramètres en *une seule passe inverse* sur le graphe, par application répétée de la règle de la chaîne.

Notation : $\delta^{(\ell)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}}$ (gradient sur la pré-activation).

Équations de la rétropropagation pour un MLP

$$\delta^{(L)} = \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot \sigma'^{(L)}(\mathbf{z}^{(L)}) \quad (\text{couche de sortie})$$

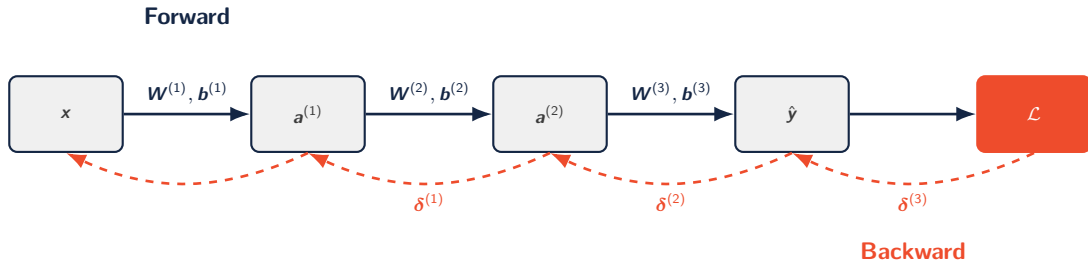
$$\delta^{(\ell)} = ((\mathbf{W}^{(\ell+1)})^{\top} \delta^{(\ell+1)}) \odot \sigma'^{(\ell)}(\mathbf{z}^{(\ell)}) \quad (\text{rétropropagation})$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = \delta^{(\ell)} (\mathbf{a}^{(\ell-1)})^{\top} \quad (\text{gradient des poids})$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \delta^{(\ell)} \quad (\text{gradient des biais})$$

$\odot$: produit de Hadamard (élément par élément).

Backpropagation — intuition graphique



Complexité : $\mathcal{O}(\#\theta)$ par batch — coût du backward $\approx 2\times$ celui du forward.

Optimiseurs et mise à jour des poids

Descente de gradient — variantes & formules

Règle de mise à jour générale

$$\theta_{t+1} = \theta_t - \eta g_t, \quad g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$$

où $\eta > 0$ est le **taux d'apprentissage** (learning rate).

Variante	Gradient utilisé	Propriétés
Batch (GD)	$g_t = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \mathcal{L}_i$	Précis, coûteux, déterministe
Stochastique (SGD)	$g_t = \nabla_{\theta} \mathcal{L}_i, i \sim \text{Unif}(N)$	Bruit utile, échappe minima locaux
Mini-batch	$g_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_{\theta} \mathcal{L}_i$	Standard en DL ($B = 32..512$)

Le mini-batch est le compromis optimal entre stabilité (gradient moyenné) et efficacité (parallélisation GPU).

Optimiseurs adaptatifs — SGD-Momentum, RMSprop, Adam

SGD avec Momentum

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t$$

$$\theta_{t+1} = \theta_t - \eta \mathbf{v}_t$$

$\beta \in [0, 1)$: accumulation des gradients passés (*inertie*).

RMSprop (Hinton, 2012)

$$\mathbf{s}_t = \rho \mathbf{s}_{t-1} + (1 - \rho) \mathbf{g}_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\mathbf{s}_t} + \varepsilon} \mathbf{g}_t$$

Adam (Kingma & Ba, 2014) — *default*

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$$

$$\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t), \quad \hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t)$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon}$$

Valeurs usuelles : $\beta_1=0.9$, $\beta_2=0.999$, $\varepsilon=10^{-8}$, $\eta=10^{-3}$.

Mise à jour des poids — exemple chiffré pour SGD

Cas d'école : neurone linéaire $\hat{y} = wx + b$, perte MSE

Données : $(x=2, y=5)$. Init : $w_0=0, b_0=0$. LR : $\eta=0.1$.

$$\text{Forward : } \hat{y}_0 = w_0x + b_0 = 0, \quad \mathcal{L}_0 = (\hat{y}_0 - y)^2 = 25$$

$$\text{Gradients : } \frac{\partial \mathcal{L}}{\partial w} = 2(\hat{y} - y)x = -20, \quad \frac{\partial \mathcal{L}}{\partial b} = 2(\hat{y} - y) = -10$$

$$\text{Update : } w_1 = w_0 - \eta \cdot (-20) = 2, \quad b_1 = b_0 - \eta \cdot (-10) = 1$$

Après 1 étape : $\hat{y}_1 = 2 \cdot 2 + 1 = 5 = y \Rightarrow \mathcal{L}_1 = 0$. **Convergé en une itération.**

Learning rate scheduling — ajuster η pendant l'entraînement

- **StepLR** : $\eta_t = \eta_0 \gamma^{\lfloor t/s \rfloor}$ — décroissance par paliers
- **CosineAnnealingLR** : $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})(1 + \cos(\pi t/T))$
- **OneCycleLR** (Smith) : warm-up + annealing $\rightarrow$ convergence rapide en fine-tuning

Régularisation & normalisation

L_2 Weight Decay

Pénalité ajoutée à la perte :

$$\mathcal{L}_{\text{tot}} = \mathcal{L} + \lambda \|\theta\|_2^2$$

Gradient modifié : $\nabla \mathcal{L}_{\text{tot}} = \nabla \mathcal{L} + 2\lambda\theta$

`optim.AdamW(weight_decay=1e-4)`

Dropout (Srivastava et al., 2014)

À l'entraînement, masquer aléatoirement :

$$\tilde{a}_i = \frac{m_i}{1-p} a_i, \quad m_i \sim \text{Bernoulli}(1-p)$$

Inactif à l'inférence (`model.eval()`).

Batch Normalization (Ioffe & Szegedy, 2015)

Normalisation par mini-batch :

$$\mu_{\mathcal{B}} = \frac{1}{B} \sum_i x_i, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{B} \sum_i (x_i - \mu_{\mathcal{B}})^2$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

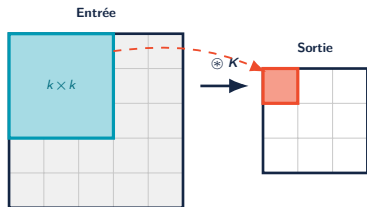
$$y_i = \gamma \hat{x}_i + \beta \quad (\gamma, \beta \text{ appris})$$

Stabilise et accélère l'entraînement (réduction du *internal covariate shift*).

PyTorch : `nn.BatchNorm2d`, `nn.LayerNorm`, `nn.GroupNorm`.

Réseaux convolutifs (CNN)

Convolution 2D — formulation, dimensions & poids



À chaque position, le noyau fait un produit scalaire ($\odot$ puis somme) $\rightarrow$ 1 pixel de sortie.

Définition discrète ($C_{in} \rightarrow C_{out}$)

Mono-canal : $(X * K)_{i,j} = \sum_{m,n} x_{i+m, j+n} K_{m,n}$

$$Y_{c,i,j} = \sum_{c'=1}^{C_{in}} \sum_{m,n} x_{c', i+m, j+n} K_{c,c',m,n} + b_c$$

Dimension de sortie

$$H_{out} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$$

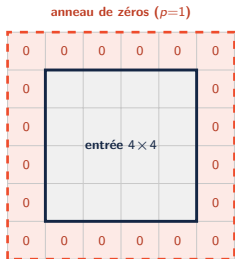
p : padding, s : stride, k : taille du noyau (idem W_{out}).

Nombre de poids d'une couche convolutive

$$\# \theta = \underbrace{C_{out} C_{in} k^2}_{\text{poids}} + \underbrace{C_{out}}_{\text{biais}}$$

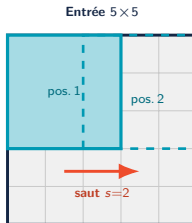
Ex. `Conv2d(3,64,3)` : $64 \cdot 3 \cdot 3^2 + 64 = 1792$ poids, **indép. de H, W .**

Convolution — padding & stride en images



Padding p — préserver la taille

On entoure l'entrée d'un **anneau de zéros**. Avec $k=3$, $p=1$, $s=1$, la sortie **garde la taille** de l'entrée (sinon elle rétrécit à chaque couche, et les bords sont sous-exploités).



Stride s — sous-échantillonner

Le noyau **saut de s cases**. $s=2 \Rightarrow \sim 2\times$ moins de positions $\Rightarrow$ sortie $\sim 2\times$ plus petite (alternative au pooling pour réduire la résolution).

Effet combiné sur la taille de sortie

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1 \quad \Rightarrow \quad \text{padding } p \uparrow \text{ agrandit, } \text{stride } s \uparrow \text{ réduit.}$$

Convolution — propriétés & receptive field

Pourquoi convolution ?

- **Partage de poids** : $C_{\text{out}} \cdot C_{\text{in}} \cdot k^2$ paramètres au lieu de $HW \cdot C$ pour un dense
- **Équivariance par translation** : un motif détecté ici l'est ailleurs
- **Localité** : exploite la structure spatiale des images

Pooling : invariance & sous-échantillonnage

- Max pooling : $y_{i,j} = \max_{(m,n) \in \mathcal{N}} x_{m,n}$
- Average pooling : $y_{i,j} = \frac{1}{|\mathcal{N}|} \sum_{(m,n) \in \mathcal{N}} x_{m,n}$
- `nn.AdaptiveAvgPool2d(1)` : sortie de taille fixe quelle que soit l'entrée

Receptive field

Taille de la région d'entrée influençant un neurone de la couche ℓ :

$$r_{\ell} = r_{\ell-1} + (k_{\ell} - 1) \prod_{i=1}^{\ell-1} s_i$$

Croît rapidement avec la profondeur $\Rightarrow$ contexte global.

Exemple — ResNet-50

Receptive field final $\approx 483 \times 483$ sur une entrée 224×224 (avec dilations et skip connections).

Architectures CNN modernes — ResNet & skip connections

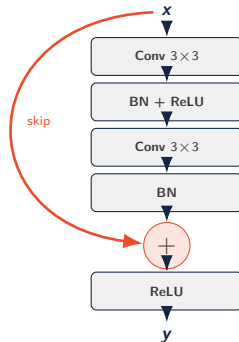
Bloc résiduel (He et al., 2015)

$$y = \mathcal{F}(x; \theta) + x$$

où $\mathcal{F} = (\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv} \rightarrow \text{BN})$.

Pourquoi ça marche ?

- Le gradient passe *directement* via le skip : $\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \left(1 + \frac{\partial \mathcal{F}}{\partial x}\right)$
- Évite le *vanishing gradient* pour les réseaux très profonds (50, 101, 152 couches)
- L'identité est un *prior* : si $\mathcal{F} \rightarrow 0$, le bloc reste utile



Métriques de performance

Matrice de confusion & métriques de classification

	Prédit +	Prédit -
Réel +	TP	FN
Réel -	FP	TN

TP/TN : justes FP/FN : erreurs

Métriques dérivées

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (\text{qualité des positifs prédits})$$

$$\text{Recall (Sensibilité)} = \frac{TP}{TP + FN} \quad (\text{couverture des vrais positifs})$$

$$\text{Specificity} = \frac{TN}{TN + FP}$$

$$F_1 = 2 \cdot \frac{\text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}} \quad (\text{moyenne harmonique})$$

ROC, AUC & métriques avancées

Courbe ROC & AUC

Tracer $TPR = \frac{TP}{TP+FN}$ vs $FPR = \frac{FP}{FP+TN}$ pour tout seuil $\tau \in [0, 1]$.

$$AUC = \int_0^1 TPR(FPR^{-1}(u)) du$$

- $AUC = 1$: classifieur parfait
- $AUC = 0.5$: aléatoire
- Insensible au déséquilibre de classes

Top-k Accuracy

$$Acc_k = \frac{1}{N} \sum_i \mathbf{1}[y_i \in \text{top}_k(\hat{y}_i)]$$

Usuelle sur ImageNet ($k=5$).

Régression

$$MSE = \frac{1}{N} \sum_i (\hat{y}_i - y_i)^2$$

$$RMSE = \sqrt{MSE}$$

$$MAE = \frac{1}{N} \sum_i |\hat{y}_i - y_i|$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

— PyTorch natif

`Accuracy`, `F1Score`, `AUROC`, `ConfusionMatrix`, `Precision`, `Recall`, `MeanAbsoluteError`...

Synthèse & transition

Synthèse mathématique du Module 1

Concept	Formule clé	PyTorch
Forward (neurone)	$a = \sigma(\mathbf{w}^\top \mathbf{x} + b)$	<code>nn.Linear</code> , <code>nn.ReLU</code>
Loss CE	$-\sum_k y_k \log \hat{y}_k$	<code>nn.CrossEntropyLoss</code>
Backprop	$\delta^{(\ell)} = (\mathbf{W}^{(\ell+1)})^\top \delta^{(\ell+1)} \odot \sigma'$	<code>loss.backward()</code>
Update SGD	$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$	<code>optim.SGD.step()</code>
Update Adam	$\theta \leftarrow \theta - \eta \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$	<code>optim.Adam</code>
Convolution	$(X * K)_{i,j} = \sum_{m,n} X_{i+m,j+n} K_{m,n}$	<code>nn.Conv2d</code>
Bloc résiduel	$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$	<code>torchvision.models.resnet50</code>
Métrique F_1	$2 \text{ P R} / (\text{P} + \text{R})$	<code>torchmetrics.F1Score</code>

Socle mathématique de tous les modules suivants : Transfer Learning, AE/VAE, GAN, ViT, Diffusion.

Du gradient au CNN.

Cap sur le Module 2 — Transfer Learning, Séquences & XAI

✉ bassem.benhamed@enetcom.usf.tn